Max Heap / Heap Sort — Source Code

```
import time

def heapify(arr, n, i):
    largest = i        # Initialize largest as root
    left = 2 * i + 1   # Left child index
    right = 2 * i + 2  # Right child index

    # Check if left child exists and is greater than root
    if left < n and arr[left] > arr[largest]:
        largest = left

    # Check if right child exists and is greater than current largest
    if right < n and arr[right] > arr[largest]:
        largest = right

    # If largest is not root, swap and continue heapifying
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)


def heap_sort(arr):
    n = len(arr)

    # Build a max heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # Extract elements one by one
    for i in range(n - 1, 0, -1):
        # Swap root (largest) with last element
        arr[0], arr[i] = arr[i], arr[0]

        # Heapify the reduced heap
        heapify(arr, i, 0)

    return arr


# --- User Input Section ---
if __name__ == "__main__":
    user_input = input("Enter numbers separated by spaces (e.g., 5 2 9 1 7): ")

    try:
        numbers = [int(x) for x in user_input.split()]

        print("\nOriginal List:", numbers)

        # Record start time
        start_time = time.perf_counter()

        # Run Heap Sort
        sorted_list = heap_sort(numbers)

        # Record end time
        end_time = time.perf_counter()

        # Calculate execution time
        execution_time = end_time - start_time

        # Print Results
        print("Sorted List:  ", sorted_list)
        print(f"Execution Time: {execution_time:.6f} seconds")

        # --- Time Complexity Info ---
        print("\n--- Time Complexity Analysis ---")
        print("Best Case: O(n log n)")
        print("Worst Case: O(n log n)")
        print("Average Case: O(n log n)")

    except ValueError:
        print("Please enter valid integers separated by spaces.")
```